Proyecto de Arquitectura del Computador

Simulación de Cache Asociativa de n vías con memoria externa

Autores

Daniel Rodulfo

Victor Pinto

Secciones

1. Contexto del proyecto
2. La librería STXXL
3. Problema
4. Técnicas computacionales
5. Comparación y discusión de resultados

Contexto

El presente proyecto tiene como finalidad la simulación de una cache asociativa por vías, desde 2, 4 hasta 8 en total, junto a la implementación de una unidad de memoria externa con la cual se realizan operaciones de entrada y salida (I/O). Todo ello con las herramientas que el lenguaje C++ ofrece, así como la utilización de la librería STXXL, la cual ofrece estructuras de datos para el manejo de la memoria externa y la expansión de los contenedores clásicos del lenguaje para que puedan procesar grandes volúmenes de datos.

Librería STXXL

Se trata de la librería fundamental que nos ofrece las herramientas y el entorno de desarrollo para simular una memoria externa de manera eficiente. Posee contenedores homólogos a los más establecidos del lenguaje C++ (vector, cola, lista) capaces de albergar y procesar una gran cantidad de datos. Aunado a esto, la librería implementa algoritmos que operan con los contenedores de manera eficiente y rápida, para alcanzar un rendimiento óptimo durante las lecturas y escrituras en el disco simulado. Entender la estructura de esta librería fue fundamental para la realización del presente proyecto.

Módulos del diseño de la librería STXXL

Interfaz STL: La cual ofrece tanto los contenedores como los algoritmos familiares a los ya presentes en C++, y con un nivel de abstracción no tan elevado.

Block Management/Manejo de Bloques: Se trata del módulo que establece el tipo de dato bloque: el tipo más básico para trabajar con la memoria externa, y el elemento que posee la información contenida en ella. Los bloques son leídos y escritos en disco. Cada vía de la cache contiene un bloque de información. Existen también procedimientos y estructuras para manejar los bloques, sus identificadores y la paginación de estos. El prefetching, una técnica para anticiparse a la carga de los bloques, es una de las implementaciones deseadas para la simulación de la Cache.

Operaciones asíncronas de I/O: Consisten en la capa concurrente de la librería que puede realizar operaciones de entrada y salida con el disco de forma eficiente, reduciendo la latencia y evitando que el procesador deba quedarse a la espera de las lecturas y escrituras.

Problema

Implementar la simulación de una Cache que puede variar en cantidad de vías de manera correcta y eficiente, utilizando las herramientas provistas por la librería STXXL, haciendo uso del disco de memoria externa y las operaciones de entrada y salida. Una cache que pue-

da detectar cuando se realizó un Miss o un Hit y operar en función de tal comportamiento, garantizando el correcto acceso a las vías, la comparación con las etiquetas, el uso del desplazamiento y la validez de una vía; así como el traslado de un bloque desde memoria a Cache.

Otro de los desafíos presentes en este proyecto es la implementación del Prefetching: una técnica que aprovecha el concepto de localidad espacial para anticipar la carga de bloques en memoria. El desarrollo de la política de reemplazo (LRU, en este caso) es también uno de los pilares del proyecto para garantizar el cumplimiento del concepto de localidad temporal.

Técnicas Computacionales

Presentaremos las distintas estructuras y técnicas empleadas para resolver los problemas planteados y cumplir con el propósito del proyecto. Dividiremos esta sección en: Algoritmos utilizados, funciones implementadas, estructuras utilizadas y ejecución del programa.

Estructuras creadas y Macros:

En primer lugar, implementamos la clase via: el objeto más pequeño dentro de una cache asociativa, y el que contiene en su interior el bloque información, una etiqueta, un bit de validez y un contador para el reloj global del programa. Esta es su definición:

```
class via {                     //Definicion del objeto via

public:

    bool validez = false;

    unsigned long long etiqueta = 0;

    unsigned long long last_uso = 0;

    block_type* dato = nullptr;

    stxxl::request_pts lectura

};
```

El tipo de dato block type, presente en la via, es el tipo de dato bloque que contiene la información. Definido junto a la Macro Block_size, se establece el tamaño de bloque a 64 Kb por bloque. Esta es su definición:

```
#define block_size (1024 * 64)
```

```
typedef stxxl::typed_block<block_size, double> block_type;
```

La librería STXXL nos brinda la creación del objeto bloque, parametrizando el tamaño y su longitud.

Nótese que en la definición de la clase via, el bit de validez es inicializado en 0. Esto nos ayuda a simular el instante de tiempo cuando se enciende un computador y comienzan las solicitudes. Así mismo, la etiqueta asignada del bloque es también 0, y el apuntador al bloque (diseñado así por como se encuentra implementado el manejo de bloques de la librería STXXL) apunta a Null.

La siguiente definición

```
typedef std::vector<via> tipo_conjunto;
```

Establece el tipo de dato conjunto. Un conjunto posee, como puede apreciarse en la declaración, la n cantidad de vías. El tipo conjunto está conformado por un vector de vías, usando el contenedor vector provisto por C++.

De manera homóloga, definimos al tipo de dato Cache como un tipo que contiene conjuntos, esto es, un vector de conjuntos.

```
typedef std::vector<tipo_conjunto> tipo_cache;
```

//Poner cantidad max de vias y cosas asi y explicar el manejo de la etiqueta

Funciones importantes:

Se agregaron funciones para el manejo de la Cache como tipo de dato. La siguiente función

```
void inicializar_cache(tipo_cache& cache, int num_conjuntos, int num_vias);
```

Es sumamente importante para la variabilidad del número de vías. El contenedor vector en el lenguaje C++, permite extender el tamaño del vector de conjuntos, así como de sus vías. Al ser inicializado, los campos de cada vía son nuevamente seteados, por convención y para que el código sea más comprensible. Al iniciar el bloque de cada vía, se crea un apuntador a tal tipo.

La siguiente función

```
void imprimir_cache(const tipo_cache& cache);
```

Simplemente muestra por pantalla el estado de la cache, de sus etiquetas y sus bits de validez. Finalmente, la siguiente función

```
void liberar_cache(tipo_cache& cache)
```

Se encarga de liberar los bloques en cada vía. Esta función se ejecuta al final del ciclo de vida

del programa.

Para la validación de las etiquetas, se usan las siguientes dos funciones:

```
int buscar_en_conjunto_paralelo(const tipo_conjunto& conjunto, unsigned long long
etiqueta_buscada)
```

```
bool coincide_etiqueta(const via& v, unsigned long long etiqueta_buscada)
```

Emplean la concurrencia, o hilos, provistos en la librería thread, para verificar en paralelo el bit de cada una de las vías de un conjunto. Esto se hace así para mantener una óptima comprobación. En el apartado de algoritmos se explicará más a detalle tales funciones.

La función

```
void simulacion(int vias, int conjuntos, Bids& bids)
```

Es la principal del programa, y se hablará de ellas en los algoritmos implementados.

Las funciones dedicadas a la política LRU:

```
int encontrar_via_lru(const tipo_conjunto& conjunto)
```

Es la función empleada para seleccionar la vía menos recientemente usada para luego ser reemplazada.

```
void actualizar_lru(tipo_conjunto& conjunto, int via, unsigned long long reloj)
```

Son las encargadas de manejar la política de acceso, menos recientemente usada, para seleccionar una vía. Se hablará de estas más adelante.

```
bool acceder_cache(tipo_conjunto& conjunto, unsigned long long etiqueta_buscada,
    unsigned long long& reloj, int& via_usada)
```

Determinará, con un conjunto y vía dadas y la etiqueta, si se produce un Hit o un Miss en una petición.

Las siguientes funciones se encargarán del cálculo de la etiqueta y de los índices de los bloques solicitados:

```
int Calcular_direccion_bloque(unsigned long long peticion)
```

Determina el número de bloque que corresponde a la dirección de bytes proporcionada, para esto se divide la dirección entre el tamaño del bloque obteniendo el índice que le corresponde a ese bloque en el vector de bids.

```
void etiqueta_indice( int direccion_bloque, int Num_conjuntos, unsigned long long
*etiqueta, int *indice)
```

Se han creado funciones variadas para calcular la dirección de cada bloque, interfaz del menú, verificar la entrada escrita por teclado y ajustar la cantidad de conjuntos a la cantidad de vías presente.

Algoritmos utilizados:

Comparación de etiquetas paralela: Se recibe en parámetros un conjunto de la cache, y la etiqueta buscada. En primera instancia, se crea un vector de resultados con el tamaño de las vías. Se crea el hilo con la librería antes mencionada. Ahora, se crea un hilo por cada vía para realizar la búsqueda. La función booleana `coincide_etiqueta` retornará verdadero si la etiqueta buscada coincide con la etiqueta en la vía actual, además de que el bit de validez debe estar activado. Se guarda el índice de la vía coincidente, y se espera a que el resto de hilos hayan concluido.

Implementación de la política LRU:

El principio de localidad temporal asegura que si un bloque ha sido accedido, es probable que en poco tiempo vuelva a serlo. Los bloques que no han sido accedidos en mucho tiempo pueden dar espacio en la Cache para otros bloques. Por lo tanto, se implementó un reloj global, que crece con cada acceso a la cache, para manejar el último uso de una vía. Se realiza una búsqueda lineal que preferiría escoger una vía disponible. Si no se encuentra una vía disponible, se opta por devolver el índice de la vía que tenga un reloj más viejo, es decir, que haya sido menos recientemente accedida.

Prefetching:

El principio de localidad espacial asegura que si un bloque fue accedido es probable que un bloque cercano al mismo sea accedido pronto, bajo esa premisa se implementó una política de prefetching en donde cuando se intenta acceder a un bloque n en la cache que resulta en un fallo al momento de buscar este bloque en la memoria se trae también el bloque siguiente al mismo (el bloque n +1), mejorando el porcentaje de aciertos, sobre todo en peticiones que siguen un patrón secuencial.

Simulación

Es el esqueleto principal del programa. Inicializa la Cache (inicialmente vacía). Un ciclo se encargará de recorrerlas. Previamente se reciben los parámetros del patrón de acceso y la cantidad de simulaciones, cada simulación consta de unas 1000 peticiones que se generan siguiendo el patrón de acceso establecido. Se calcula tanto la etiqueta como el índice de cada bloque, y se accede a la cache con el conjunto actual y la vía, usando la función

```
bool acceder_cache(tipo_conjunto& conjunto, unsigned long long etiqueta_buscada,
    unsigned long long& reloj, int& via_usada)
```

que será descrita próximamente. La función devolverá verdadero si efectivamente hubo un Hit, y se acumularán la cantidad de Hits dados. Caso contrario, hubo un Miss y se cuenta. Usando uno de los procedimientos de la librería STXXL, se carga el bloque a la Cache usando la función

```
void cargar_bloque(via& v,const block_type::bid_type& bid)
```

La cual primero comprueba si la vía no tiene una petición de lectura en proceso, que en dado caso de tenerla espera a que este proceso termine y luego manda la petición de lectura del nuevo bloque solicitado.

```
bool acceder_cache(tipo_conjunto& conjunto, unsigned long long etiqueta_buscada,
    unsigned long long& reloj, int& via_usada)
```

es la encargada de arrojar si se produjo un Hit o un Miss. Al principio, se utiliza la función `buscar_en_conjunto_paralelo(conjunto, etiqueta_buscada)` para verificar si la etiqueta se encuentra en el conjunto actual. De ser así, la función padre devolverá un Hit inmediatamente. Se actualizará el uso de la vía empleada para la política LRU, usando la función `void actualizar_lru(tipo_conjunto& conjunto, int via, unsigned long long reloj)` que marcará la vida usada como la más reciente.

Si hubo un fallo, se usará la función

```
int encontrar_via_lru(const tipo_conjunto& conjunto)
```

Que encontrará la vía menos recientemente usada, o least recently used (LRU). Esto refuerza el principio de localidad temporal. Se busca linealmente en el conjunto dado. Si se encuentra una vía libre se escoge, sino, se devolverá la vía que menos fue usada. Nótese que a cada acceso en la cache, el reloj global creado aumenta, y este es utilizado para, al momento de actualizar el uso de una vía particular, se le modifique la última vez que fue usada. Una vez retornada la vía a reemplazar (o en la que se va a escribir si esta no estaba inicializada), se procede a escribir el bloque 'n' solicitado y siguiendo la lógica de prefetching previamente establecida se carga también el bloque 'n+1' si y solo si 'n+1' está dentro del rango del número de bloques establecidos. Finalizada la simulación se guarda el porcentaje de aciertos y fallos de la misma y se suma al de las simulaciones previas para al final de todas las simulaciones sacar un promedio entre todas las simulaciones hechas.

Discusión de resultados

Para los casos de prueba se diseñaron 3 patrones de acceso: aleatorio, secuencial y coincidencia (forzamos varias peticiones de acceso en secuencia a un mismo conjunto).

Aleatorio: Se observó que para conjuntos ya sea de 2, 4 y 8 vías con peticiones aleatorias que no sigan ningún patrón el porcentaje de aciertos y fallos queda de la siguiente manera de forma consistente para las 3 configuraciones de vías Hits: 15-16 % Misses: 84-85 %, se probó eliminar el prefetching para observar el efecto para este tipo de acceso pero el porcentaje de aciertos y misses fue el mismo.

Secuencial: Para el patrón de acceso secuencial se decidió programar peticiones que van avanzando 4kb en cada iteración hasta que dan la vuelta. Para este patrón se observó consistentemente un porcentaje de aciertos y fallos para las 3 configuraciones de .Aciertos: 96.8 %

Fallos: 3.2 %". Se probó a eliminar el prefetching para observar si había una disminución en la tasa de aciertos y efectivamente, la tasa de fallos se duplicó consistentemente para todas las configuraciones de vías, sin el prefetching el porcentaje pasó a ser de .ᴬciertos: 93.7 % Fallos: 6.3 %", mostrando como el prefetching puede influir positivamente para esta clase de patrones de acceso.

Coincidencia: Para este acceso se generó un patrón de direcciones que fuerza a que desde la primera iteración se repitan 6 bloques cuyas direcciones corresponden a número de bloques definidas por un factor de 8 (0, 8, 16, ...) de forma cíclica. Con este patrón de acceso pudimos observar una tasa de aciertos en las configuraciones 2 y 4 vías de .ᴬciertos: 0 % Fallos: 100 %", ya que todos los bloques pertenecían al mismo conjunto la cantidad de vías no permitía que los tuviese todos causando que nunca pudiesen dar un acierto, en cambio con la configuración de 8 vías la tasa de aciertos fue de .ᴬciertos: 99.4 % Fallos: 0.6 %". Acá se pudo ver el valor de asociatividad, a medida que la misma aumenta disminuye la cantidad de fallos ante peticiones consecutivas que correspondan a un mismo conjunto. Para esta prueba también se probó a eliminar el prefetching, pero por la misma metodología empleada los resultados fueron exactamente los mismos, ya que entre cada bloque habían 8 bloques de separación el prefetching nunca predecía la carga que correspondería a la petición siguiente.